

E-book · Encontro 1: Fundamentos de IHC, Usabilidade e Acessibilidade

Professor Dr. Marcelo Amorim

Apresentação

Este e-book aprofunda os temas do Encontro 1 da disciplina **Usabilidade, Desenvolvimento Web, Mobile e Jogos**: os fundamentos da Interação Humano-Computador (IHC), o conceito de usabilidade e a acessibilidade digital. Ele foi pensado como material de apoio para ler depois da aula, retomando o que vimos e indo um pouco além.

A ideia central que atravessa todos os capítulos é simples: um sistema só funciona de verdade quando as pessoas conseguem usá-lo. Um código correto, performático e bem testado ainda pode falhar se quem o usa não entende o que fazer, não percebe o que aconteceu ou simplesmente não consegue interagir com ele.

Como o material está organizado. Os capítulos 1 a 4 tratam dos fundamentos de IHC: o que é a área, modelos mentais, affordances e signifiers, e os golfs de execução e avaliação. Os capítulos 5 e 6 tratam de usabilidade: sua definição, seus atributos e como avaliá-la. Os capítulos 7 a 9 tratam de acessibilidade: conceito, contexto brasileiro, as diretrizes WCAG e sua aplicação em web, mobile e jogos. O capítulo 10 mostra como esses temas se encaixam em algo maior, a experiência do usuário. Ao final, há uma síntese, um glossário e questões para reflexão.

Como aproveitar melhor. Leia com um navegador aberto ao lado. Sempre que um conceito aparecer, tente encontrá-lo em um site, app ou jogo que você usa. Nos capítulos de acessibilidade, teste os exemplos de verdade: navegue só com o teclado, ative o leitor de tela do seu celular, aumente o tamanho da fonte do sistema. A compreensão muda bastante quando você experimenta.

1. O que é Interação Humano-Computador

Interação Humano-Computador (IHC) é a área que estuda o projeto, a avaliação e a implementação de sistemas computacionais interativos para uso humano, além dos fenômenos que cercam esse uso. Essa definição, baseada na proposta da ACM SIGCHI, destaca três atividades: projetar, avaliar e construir. As três interessam diretamente a quem desenvolve software.

Uma área interdisciplinar

IHC nasce do encontro de várias disciplinas, e cada uma responde a uma pergunta diferente:

Área	Pergunta que ajuda a responder	Exemplo de contribuição
Ciência da Computação	Como construir a interface?	Frameworks, padrões de arquitetura, desempenho
Psicologia cognitiva	Como as pessoas percebem, lembram e decidem?	Limites de memória, atenção, modelos mentais
Design	Como dar forma e comunicar?	Hierarquia visual, tipografia, cor, layout
Ergonomia	Como adaptar ao corpo e ao contexto?	Tamanho de alvos de toque, postura, fadiga
Linguística e comunicação	Como a interface "conversa" com quem usa?	Terminologia, mensagens, microtextos

Essa variedade explica por que IHC não é "só deixar bonito". A beleza importa, mas é apenas uma parte de um problema maior: fazer com que pessoas reais consigam atingir seus objetivos usando o sistema.

Por que IHC importa para quem desenvolve

Para quem usa, a interface é o sistema. A pessoa não vê o banco de dados, a API ou a arquitetura de microsserviços. Ela vê telas, botões, mensagens e respostas. Se essa camada falha, todo o resto fica invisível.

Alguns exemplos nas três plataformas da disciplina:

- **Web:** o back-end processa o pedido corretamente, mas o botão "Finalizar compra" fica escondido no fim de uma página longa, e muitas pessoas desistem antes de encontrá-lo.
- **Mobile:** a API responde em poucos milissegundos, mas o app não mostra nenhum indicador de carregamento. A pessoa acha que o toque não funcionou, toca de novo e acaba enviando o pedido três vezes.
- **Jogos:** a mecânica de abrir baús funciona perfeitamente, mas o baú se confunde com o cenário. O jogador passa ao lado dele sem perceber que pode interagir.

Em todos os casos, os testes automatizados passariam. O problema só aparece quando uma pessoa tenta usar o sistema. Por isso, IHC complementa a engenharia de software: enquanto os testes verificam se o sistema faz o que foi especificado, a IHC pergunta se aquilo que foi especificado faz sentido para quem vai usar.

O custo de ignorar a interface

Problemas de interface têm custo real. Em sistemas comerciais, viram abandono de carrinho, avaliações ruins nas lojas de aplicativos e chamados de suporte. Em sistemas internos, viram retrabalho e erros operacionais. Em sistemas críticos, como os de saúde ou aviação, um erro de interface pode ter consequências graves. Além disso, corrigir um problema de interface depois que o sistema está pronto costuma ser muito mais caro do que preveni-lo no projeto.

2. Modelos mentais

As pessoas agem com base no que acham que o sistema faz, e não no que ele realmente faz. Essa representação interna, construída a partir da experiência, da observação e de analogias com outras coisas conhecidas, é chamada de **modelo mental**.

Modelos mentais costumam ser incompletos e, às vezes, errados. Isso não é um problema em si: ninguém precisa entender como funciona um motor para dirigir um carro. O problema surge quando o modelo mental leva a pessoa a agir de um jeito que o sistema não espera.

Três modelos em jogo

Don Norman, em *O Design do Dia a Dia*, propõe olhar para três modelos diferentes:

- **Modelo do projetista:** como quem projetou e desenvolveu entende o sistema. Inclui detalhes de implementação que o usuário nunca verá.
- **Imagem do sistema:** tudo o que o sistema mostra de si mesmo: a interface, as mensagens, o comportamento, a documentação. É o único canal de comunicação entre projetista e usuário.
- **Modelo do usuário:** como a pessoa que usa entende o sistema, construído a partir da imagem do sistema e de experiências anteriores.

O ponto crucial é que o projetista não conversa diretamente com o usuário. Tudo o que ele quer comunicar precisa estar na imagem do sistema. Se a interface não transmite um modelo coerente, cada pessoa vai construir o seu próprio, muitas vezes errado.

Por que desenvolvedores têm um ponto cego

Quem desenvolve conhece a implementação. Por isso, tende a achar óbvio o que, para quem usa, não é. Veja um exemplo de sincronização de arquivos na nuvem:

Perspectiva	Como entende o que aconteceu
Quem desenvolveu	O arquivo foi sincronizado com o armazenamento remoto e a cópia local foi substituída por um atalho para economizar espaço.
Quem usa	Meu arquivo sumiu? Ele está no computador ou na internet? Se eu ficar sem conexão, consigo abrir?

As duas visões são legítimas, mas a interface precisa respeitar a segunda. Uma boa solução seria mostrar ícones claros indicando se o arquivo está disponível offline e explicar isso na primeira vez que acontecer.

Exemplos nas três plataformas

- **Web:** muitas pessoas acreditam que o botão "Voltar" do navegador desfaz a última ação. Em formulários de várias etapas feitos como aplicações de página única, voltar pode levar para fora do fluxo e apagar tudo o que foi preenchido.
- **Mobile:** usuários costumam achar que fechar um app pelo gerenciador de tarefas economiza bateria, porque esse foi o modelo construído em sistemas antigos. Nos sistemas atuais, isso pode até aumentar o consumo, já que o app precisa ser carregado do zero depois.
- **Jogos:** um jogador que vem de outro jogo do mesmo gênero traz um modelo mental pronto: espera que o mesmo botão pule, que vermelho indique perigo, que o mapa abra com a mesma tecla. Quebrar essas convenções sem um bom motivo gera frustração.

Como trabalhar a favor dos modelos mentais

Três estratégias ajudam. A primeira é seguir convenções já conhecidas, como o carrinho no canto superior direito de uma loja virtual ou o ícone de engrenagem para configurações. A segunda é usar metáforas coerentes, como a lixeira, que comunica "apagado, mas recuperável". A terceira é dar feedback que confirme ou corrija o modelo da pessoa, mostrando claramente o que aconteceu depois de cada ação.

3. Affordances, signifiers, mapeamento e feedback

Uma interface bem projetada comunica, sem precisar de manual, o que pode ser feito, onde e como. Don Norman descreve quatro conceitos que ajudam a analisar essa comunicação.

Affordance

Affordance é a relação entre as propriedades de um objeto e as capacidades de quem o usa, que determina quais ações são possíveis. Uma cadeira permite sentar; um botão físico permite apertar; uma alça permite segurar. A affordance não é uma propriedade do objeto sozinho: uma cadeira alta demais não oferece a affordance de sentar para uma criança pequena.

Signifier

Signifier (em português, às vezes traduzido como significante) é qualquer sinal perceptível que comunica onde e como agir. A affordance diz o que é possível; o signifier mostra isso para quem usa.

Norman passou a enfatizar os signifiers porque, no design digital, o termo affordance era muito usado de forma imprecisa. Em uma tela, praticamente tudo pode ser clicado: o que importa é se a pessoa percebe o que é clicável.

O exemplo clássico são as portas. Um puxador vertical sinaliza "puxe"; uma placa lisa de metal sinaliza "empurre". Quando a porta tem um puxador mas precisa ser empurrada, o signifier contradiz a ação correta, e alguém acaba colando uma placa "EMPURRE". Sempre que uma etiqueta improvisada é necessária, é sinal de que o design não comunicava.

Signifiers no digital

Plataforma	Signifier ausente ou ambíguo	Signifier claro
Web	Texto solto "Enviar formulário", sem forma de botão	Botão com fundo, borda, contraste e cursor de mãozinha
Web	Link com a mesma cor do texto e sem sublinhado	Link sublinhado ou com cor distinta e consistente

Plataforma	Signifier ausente ou ambíguo	Signifier claro
Mobile	Item de lista que pode ser deslizado, sem nenhuma pista	Dica visual "← arquivar" ou animação sutil no primeiro uso
Mobile	Ícone desconhecido sem rótulo	Ícone acompanhado de texto curto
Jogos	Objeto interativo igual aos objetos decorativos	Contorno, brilho ou indicação do botão ao se aproximar

O chamado design "flat", que remove sombras, bordas e relevos, deixa a interface mais limpa, mas pode eliminar signifiers importantes. Não há problema em interfaces minimalistas, desde que os elementos interativos continuem reconhecíveis.

Mapeamento

Mapeamento é a relação entre os controles e seus efeitos. Um bom mapeamento é natural: os controles estão dispostos de forma que corresponde ao que controlam. O exemplo clássico é o fogão: quando os botões ficam em linha e as bocas em quadrado, ninguém sabe qual botão acende qual boca. O mesmo vale para um painel de interruptores idênticos, todos iguais e sem indicação.

No digital, um bom mapeamento aparece quando a barra de rolagem se move no mesmo sentido do conteúdo, quando o controle de volume sobe para aumentar ou quando, em um jogo, o analógico para a direita move o personagem para a direita.

Feedback

Feedback é a resposta do sistema a cada ação, informando o que aconteceu. Deve ser imediato, claro e proporcional. Um botão que muda de aparência ao ser pressionado, um indicador de progresso durante o envio, uma vibração curta no celular ou um som quando o jogador coleta um item são formas de feedback.

A ausência de feedback é uma das causas mais comuns de erro. Sem resposta, a pessoa repete a ação, desiste ou presume que algo deu errado. O excesso também atrapalha: notificações constantes e animações longas para ações triviais viram ruído.

4. Os golfos de execução e de avaliação

Toda interação com um sistema envolve duas travessias: ir da intenção à ação e voltar do resultado à compreensão. Norman chama as dificuldades dessas travessias de **golfos**.

Golfo de execução

É a distância entre o que a pessoa quer fazer e as ações que o sistema oferece. A pergunta típica é: *como eu faço o que quero?* O golfo é grande quando a função existe, mas a pessoa não sabe onde está ou como acioná-la. Menus escondidos, comandos que só existem como atalho de teclado e gestos sem nenhuma indicação aumentam esse golfo.

Golfo de avaliação

É a distância entre o que o sistema fez e o que a pessoa consegue perceber e interpretar. A pergunta típica é: *o que aconteceu? Deu certo?* O golfo é grande quando falta feedback, quando o feedback é ambíguo ou quando ele usa uma linguagem que a pessoa não entende.

Os sete estágios da ação

Norman detalha a interação em sete estágios, que formam um ciclo:

1. **Objetivo:** o que a pessoa quer alcançar (por exemplo, "enviar o trabalho para o professor").
2. **Planejamento:** decidir qual caminho seguir ("vou usar o ambiente virtual da disciplina").
3. **Especificação:** definir as ações concretas ("abrir a tarefa, clicar em anexar, escolher o arquivo").
4. **Execução:** realizar as ações na interface.
5. **Percepção:** notar o que mudou no sistema.
6. **Interpretação:** entender o que essa mudança significa.
7. **Comparação:** verificar se o objetivo foi atingido.

Os estágios 2 a 4 atravessam o golfo de execução; os estágios 5 a 7 atravessam o golfo de avaliação. Se a pessoa não atingiu o objetivo, o ciclo recomeça. O modelo é útil para localizar onde exatamente a interação falhou.

No exemplo do envio de trabalho, imagine que o arquivo foi anexado, mas a tarefa só é entregue depois de um segundo clique em "Enviar para avaliação", que fica no fim da página. A pessoa vê o arquivo listado (percepção), conclui que entregou (interpretação) e fecha a página. O erro está no golfo de avaliação: o sistema não deixou claro que a entrega ainda não tinha sido concluída.

Aplicação: mensagens de erro

Mensagens de erro são um bom lugar para observar os golfos. Compare:

Mensagem	Golfo de avaliação	Golfo de execução
"Erro. Falha na operação (código 0x80070005)." [OK]	Grande: a pessoa não sabe o que houve	Grande: não há indicação do que fazer a seguir
"Não foi possível salvar o arquivo. Você não tem permissão para salvar nesta pasta." [Escolher outra pasta] [Cancelar]	Pequeno: a causa está explicada em linguagem comum	Pequeno: a próxima ação está disponível no próprio diálogo

Uma boa mensagem de erro diz o que aconteceu, por que aconteceu e como resolver, sem culpar a pessoa. Códigos técnicos podem aparecer, mas como informação complementar para o suporte, nunca como a única explicação.

Como reduzir os golfos

Para o golfo de execução: deixar as ações visíveis, usar signifiers claros, seguir convenções e organizar a interface de acordo com as tarefas das pessoas, e não com a estrutura do banco de dados. Para o golfo de avaliação: dar feedback imediato e claro, mostrar o estado atual do sistema e usar a linguagem de quem usa.

5. O que é usabilidade

Usabilidade é a medida de quão bem pessoas específicas conseguem usar um sistema para atingir seus objetivos. Ela não é uma característica absoluta do produto: depende de quem usa, para quê e em que situação.

A definição da ISO 9241-11

A norma ISO 9241-11 define usabilidade como o grau em que um sistema, produto ou serviço pode ser usado por **usuários específicos** para alcançar **objetivos específicos** com eficácia, eficiência e satisfação, em um **contexto de uso específico**.

As três dimensões centrais são:

Dimensão	Pergunta	Como costuma ser medida
Eficácia	A pessoa conseguiu concluir a tarefa, com precisão?	Taxa de sucesso nas tarefas, quantidade de erros
Eficiência	Quanto esforço e recursos foram gastos?	Tempo na tarefa, número de cliques ou passos
Satisfação	Como a pessoa se sentiu ao usar?	Questionários como o SUS, entrevistas

As três expressões em destaque na definição são fundamentais. Um sistema de controle de tráfego aéreo pode ser altamente usável para controladores treinados e totalmente inutilizável para um leigo. Um app de entregas pode funcionar bem em casa, com Wi-Fi e as duas mãos livres, e mal na rua, com sol forte e uma sacola na mão. Por isso, dizer que algo "tem boa usabilidade" só faz sentido quando se sabe para quem e em qual contexto.

Essa ideia de "usuários específicos" é também a ponte para a acessibilidade: se o público inclui pessoas com deficiência, um sistema que elas não conseguem usar não tem boa usabilidade para esse público.

Os cinco atributos de Nielsen

Jakob Nielsen, em *Usability Engineering* (1993), descreve a usabilidade por meio de cinco atributos. Eles complementam a visão da ISO, detalhando aspectos que aparecem ao longo do tempo de uso:

Atributo	Pergunta central	Exemplo de problema
Facilidade de aprendizado	Quão fácil é realizar tarefas básicas no primeiro contato?	App de banco que exige descobrir sozinho onde fica o Pix
Eficiência de uso	Depois de aprender, quão rapidamente as tarefas são feitas?	Sistema interno sem atalhos, que obriga a percorrer cinco telas para uma tarefa diária
Facilidade de memorização	Ao voltar após um tempo, quão fácil é reaprender?	Sistema da declaração de imposto, usado uma vez por ano, que parece novo a cada vez
Baixa taxa de erros	Quantos erros acontecem,	Formulário que apaga

Atributo	Pergunta central	Exemplo de problema
	quão graves são e quão fácil é se recuperar?	todos os campos quando um deles está inválido
Satisfação	Quão agradável é usar o sistema?	Jogo com menus lentos e confusos que irritam mesmo quem gosta da mecânica

Tensões entre atributos

Os atributos nem sempre caminham juntos. Interfaces muito guiadas, com assistentes passo a passo, favorecem o aprendizado, mas podem atrapalhar a eficiência de quem já é experiente. Atalhos de teclado aumentam a eficiência, mas não ajudam quem está começando.

A solução costuma ser oferecer caminhos para os dois perfis. IDEs de programação são um bom exemplo: tudo está acessível pelos menus, para quem está aprendendo, e também por atalhos e paleta de comandos, para quem já domina. Em jogos, o mesmo princípio aparece em tutoriais que podem ser pulados e em opções de dificuldade.

6. Como avaliar a usabilidade

Usabilidade se avalia de duas grandes formas: inspecionando a interface com base em conhecimento especializado, ou observando pessoas reais usando o sistema. Os dois caminhos se complementam.

Métodos de inspeção

Nos métodos de inspeção, especialistas analisam a interface sem a participação de usuários, usando princípios, diretrizes ou roteiros. São mais rápidos e baratos e podem ser aplicados cedo, até mesmo em protótipos de papel.

O método mais conhecido é a **avaliação heurística**, proposta por Nielsen, em que avaliadores percorrem a interface verificando se ela respeita um conjunto de princípios gerais, as heurísticas. Esse será o tema do próximo encontro. Outro exemplo é o **percurso cognitivo**, em que o avaliador simula o raciocínio de um usuário novato, passo a passo, perguntando se ele saberia o que fazer em cada etapa.

A limitação principal da inspeção é que o especialista não é o usuário. Ele pode deixar passar problemas que surgem do contexto real, do vocabulário do público ou de expectativas que não previu.

Testes com usuários

No **teste de usabilidade**, pessoas representativas do público realizam tarefas realistas enquanto são observadas. É o método que revela, com mais segurança, onde as pessoas realmente têm dificuldade.

Um teste típico tem alguns elementos:

- **Tarefas em forma de cenário:** descrevem um objetivo sem revelar o caminho. "Você vai viajar amanhã e quer saber o horário do último ônibus para o centro" funciona; "clique em Linhas e procure o horário" não, porque já entrega a resposta.
- **Protocolo de pensar em voz alta:** a pessoa verbaliza o que está procurando, o que espera e o que a surpreende. Isso torna visível o modelo mental dela.
- **Moderador neutro:** quem conduz não ajuda, não explica a interface e não induz respostas. Perguntas como "o que você esperava que acontecesse?" são mais úteis do que dicas.
- **Registro:** anotações, gravação de tela e, quando possível, métricas.

Um ponto importante para quem conduz: o que está sendo testado é o sistema, não a pessoa. Dizer isso explicitamente no início deixa o participante mais à vontade para errar, e os erros são justamente o que revela os problemas.

Não é preciso muita gente. Nielsen popularizou a ideia de que testes pequenos, com cerca de cinco participantes por rodada, já revelam a maior parte dos problemas mais frequentes. Melhor do que um teste grande é fazer vários testes pequenos ao longo do desenvolvimento, corrigindo os problemas entre uma rodada e outra.

Métricas comuns

Métrica	O que indica	Dimensão da ISO
Taxa de sucesso	Porcentagem de participantes que concluem a tarefa	Eficácia
Número de erros	Quantas vezes a pessoa toma um caminho errado	Eficácia
Tempo na tarefa	Quanto tempo leva para concluir	Eficiência
SUS (System Usability Scale)	Questionário de 10 itens que gera uma pontuação de 0 a 100	Satisfação

Quando usar cada abordagem

Inspeção é indicada para encontrar rapidamente problemas evidentes, especialmente no início do projeto ou quando há pouco tempo e orçamento. Testes com usuários são indicados para validar decisões importantes e descobrir problemas que só aparecem no uso real. Em projetos bem conduzidos, a inspeção elimina os problemas óbvios antes, para que os testes com usuários rendam mais.

7. Acessibilidade: conceito e contexto

Acessibilidade digital é a garantia de que pessoas com deficiência possam perceber, compreender, navegar e interagir com sistemas digitais, com autonomia. Ela não é um recurso extra nem um "modo especial": é uma condição para que o sistema funcione para todo o seu público.

Tipos de deficiência e tecnologias assistivas

As barreiras digitais variam conforme o tipo de deficiência, e cada uma se relaciona a tecnologias assistivas diferentes:

Tipo	Exemplos de barreira	Tecnologias e estratégias usadas
Visual (cegueira)	Imagens sem descrição, botões sem nome, layout que só faz sentido visualmente	Leitores de tela (NVDA, JAWS, VoiceOver, TalkBack), linha braille
Visual (baixa visão, daltonismo)	Baixo contraste, texto pequeno, informação transmitida só por cor	Ampliação, alto contraste, ajuste de fonte
Auditiva	Vídeos sem legenda, alertas apenas sonoros	Legendas, transcrições, Libras, alertas visuais
Motora	Alvos pequenos, ações que exigem precisão ou rapidez, dependência do mouse	Teclado, acionadores, controle por voz, rastreamento ocular
Cognitiva e de aprendizagem	Textos complexos, excesso de informação, limites de tempo	Linguagem simples, estrutura clara, tempo ajustável

Um ponto que costuma surpreender: usuários experientes de leitores de tela navegam com muita rapidez, ouvindo a fala em velocidades bem acima da conversação normal. Quando a navegação fica lenta ou confusa, o problema quase sempre está no site, não na pessoa.

Deficiências permanentes, temporárias e situacionais

A acessibilidade beneficia muito mais gente do que parece. O *Inclusive Design Toolkit*, da Microsoft, propõe olhar para as limitações em três níveis:

Capacidade	Permanente	Temporária	Situacional
Visão	Pessoa cega	Pupila dilatada após exame	Tela sob sol forte
Movimento	Pessoa sem um braço	Braço engessado	Segurando um bebê no colo
Audição	Pessoa surda	Infecção no ouvido	Ambiente barulhento

Uma solução pensada para uma deficiência permanente costuma ajudar todos os outros grupos. Legendas foram criadas para pessoas surdas e hoje são usadas por quem assiste vídeos no transporte público sem fone. Bom contraste ajuda pessoas com baixa visão e também quem usa o celular ao ar livre.

O contexto brasileiro

Segundo a PNAD Contínua 2022 do IBGE, cerca de 18,6 milhões de pessoas com 2 anos ou mais declararam ter alguma deficiência no Brasil, o equivalente a aproximadamente 8,9% dessa população. Ignorar acessibilidade significa excluir um público do tamanho de um país inteiro. (Vale conferir os números atualizados no site do IBGE.)

Além da questão ética, há obrigação legal:

- **Lei Brasileira de Inclusão (Lei nº 13.146/2015):** também chamada de Estatuto da Pessoa com Deficiência, torna obrigatória a acessibilidade nos sítios mantidos por empresas com sede ou representação comercial no país e por órgãos de governo (art. 63).
- **eMAG (Modelo de Acessibilidade em Governo Eletrônico):** conjunto de recomendações para os sites do governo federal, alinhado às diretrizes internacionais WCAG.

Para quem vai trabalhar com desenvolvimento, isso significa que acessibilidade é requisito, não um diferencial opcional. E, como acontece com a usabilidade, é muito mais barato incorporá-la desde o início do que corrigir depois.

8. WCAG e os quatro princípios

As **WCAG (Web Content Accessibility Guidelines)**, publicadas pelo W3C, são a principal referência internacional de acessibilidade digital, atualmente na versão 2.2. Elas se organizam em quatro princípios, que em inglês formam a sigla POUR. Embora o nome fale em "web", os princípios servem de base também para apps e jogos.

Níveis de conformidade

Cada critério das WCAG tem um nível. O **A** reúne o mínimo indispensável; sem ele, alguns grupos simplesmente não conseguem usar o conteúdo. O **AA** é o nível mais usado como referência em leis, políticas e contratos. O **AAA** é o mais rigoroso e nem sempre é viável para todo tipo de conteúdo.

1. Perceptível

A informação e os componentes da interface precisam ser apresentados de formas que as pessoas consigam perceber, por mais de um sentido quando necessário.

Texto alternativo. Imagens que transmitem informação precisam de uma descrição que comunique o mesmo conteúdo:

```
<!-- Sem descrição: o leitor de tela anuncia apenas "imagem" ou o nome do
arquivo -->

```

```
<!-- Com descrição: comunica o que a imagem mostra -->

```

```
<!-- Imagem decorativa: alt vazio para o leitor de tela ignorar -->

```

O texto alternativo deve descrever a função ou a informação, não a aparência. "Imagem de um gráfico" não ajuda; "Vendas cresceram 20% no segundo trimestre" ajuda.

Contraste. No nível AA, texto normal precisa de contraste mínimo de 4,5:1 em relação ao fundo, e texto grande, de 3:1. Ferramentas como o DevTools do navegador e o WebAIM Contrast Checker calculam isso.

Não depender só de cor. Um campo com erro marcado apenas em vermelho é invisível para muitas pessoas com daltonismo. Acrescente ícone e texto ("E-mail inválido").

Mídia. Vídeos precisam de legendas, e áudios, de transcrição.

2. Operável

Os componentes precisam poder ser operados de diferentes formas, não apenas com o mouse.

Teclado. Tudo o que funciona com o mouse precisa funcionar com o teclado: Tab avança, Shift+Tab volta, Enter e Espaço ativam, Esc fecha. A ordem de navegação deve seguir a ordem lógica da página.

Foco visível. Quem navega por teclado precisa ver onde está. Remover o contorno de foco por estética é um dos erros mais comuns em CSS:

```
/* Nunca faça isso */
:focus { outline: none; }

/* Faça isso: o foco aparece na navegação por teclado */
:focus-visible {
  outline: 3px solid #1D5FC2;
  outline-offset: 2px;
}
```

Alvos de toque. Áreas pequenas prejudicam todo mundo, especialmente pessoas com tremores. A Apple recomenda alvos de pelo menos 44 pt e o Material Design, 48 dp.

Tempo e movimento. Limites de tempo precisam poder ser estendidos, e conteúdos que piscam mais de três vezes por segundo devem ser evitados, pois podem provocar crises em pessoas com epilepsia fotossensível.

3. Compreensível

O conteúdo e o funcionamento da interface precisam ser claros e previsíveis.

Rótulos em formulários. Todo campo precisa de um rótulo associado. O placeholder não substitui o rótulo, porque desaparece quando a pessoa começa a digitar e costuma ter pouco contraste.

```
<!-- Ruim: sem rótulo, só placeholder -->
<input placeholder="E-mail">

<!-- Bom: rótulo associado ao campo -->
<label for="email">E-mail</label>
<input id="email" type="email" autocomplete="email">
```

Erros úteis. Mensagens de erro devem dizer o que está errado e como corrigir, retomando o que vimos sobre o golfo de avaliação.

Comportamento previsível. Elementos que fazem a mesma coisa devem ter a mesma aparência e o mesmo nome em todo o sistema, e a interface não deve mudar de contexto sozinha, sem aviso.

4. Robusto

O conteúdo precisa funcionar com diferentes navegadores e tecnologias assistivas, hoje e no futuro. Na prática, isso significa usar HTML semântico, que já carrega significado para as tecnologias assistivas:

```
<!-- Ruim: não recebe foco, não responde a Enter e não é anunciado como botão -->  
<div onclick="enviar()">Enviar</div>
```

```
<!-- Bom: foco, teclado e papel de botão já incluídos -->  
<button type="submit">Enviar</button>
```

E quanto ao **ARIA**? Os atributos ARIA (como `aria-label` e `aria-expanded`) complementam o HTML quando não existe um elemento nativo adequado, por exemplo em componentes personalizados como abas ou menus complexos. A própria orientação do W3C recomenda usar primeiro o elemento HTML nativo. ARIA mal aplicado pode deixar a experiência pior do que nenhum ARIA.

Como verificar

Ferramentas automáticas como Lighthouse, WAVE e axe DevTools encontram rapidamente problemas como falta de texto alternativo e contraste insuficiente. Mas elas detectam apenas uma parte dos problemas. A verificação manual continua indispensável: navegar a página inteira só com teclado, testar com um leitor de tela e ampliar a página para 200%.

9. Acessibilidade em mobile e jogos

Os princípios POUR valem para qualquer plataforma, mas mobile e jogos têm particularidades que merecem atenção própria.

Mobile

Os sistemas Android e iOS oferecem recursos de acessibilidade nativos, e os apps precisam funcionar bem com eles:

- **Leitores de tela:** TalkBack (Android) e VoiceOver (iOS). Funcionam por gestos: deslizar para a direita passa para o próximo elemento, toque duplo ativa. Todo elemento interativo precisa ter um nome compreensível, o que exige rótulos em ícones sem texto (`contentDescription` no Android, `accessibilityLabel` no iOS e no React Native, `Semantics` no Flutter).
- **Tamanho de fonte do sistema:** muitas pessoas aumentam a fonte nas configurações do celular. Apps que usam tamanhos fixos ignoram essa

preferência; apps que respeitam, mas não foram testados, costumam ter textos cortados e botões sobrepostos.

- **Alvos de toque:** como vimos, 44 pt no iOS e 48 dp no Android, com espaço entre eles.
- **Orientação de tela:** o app deve funcionar em retrato e paisagem, salvo quando a orientação for essencial. Algumas pessoas usam o celular fixo em suportes de cadeira de rodas, em uma posição só.
- **Gestos alternativos:** ações que dependem de gestos complexos (pinça, deslizar com dois dedos) precisam de uma alternativa simples, como um botão.

Como testar. Ative o leitor de tela do seu próprio celular e tente completar uma tarefa no app que você está desenvolvendo. Depois, aumente a fonte do sistema ao máximo e percorra as telas principais. No Android, o app gratuito Accessibility Scanner analisa a tela e aponta problemas como alvos pequenos e baixo contraste.

Jogos

Jogos colocam desafios específicos, porque dependem muito de reação rápida, coordenação, percepção visual e sonora. Ao mesmo tempo, a indústria evoluiu bastante nos últimos anos, e vários títulos de grande porte oferecem menus de acessibilidade completos.

Área	Recursos de acessibilidade
Visão	Modos para daltonismo, informações que não dependem só de cor, interface ampliável, alto contraste
Audição	Legendas legíveis com indicação de quem fala, pistas visuais para sons importantes (passos, tiros, alertas)
Motora	Remapeamento de controles, alternativa a apertar botões repetidamente, opção de segurar em vez de pressionar rápido
Cognitiva	Opções de dificuldade, tutoriais que podem ser revistos, lembretes do objetivo atual

As **Game Accessibility Guidelines** (gameaccessibilityguidelines.com) reúnem recomendações organizadas em três níveis de esforço: básico, intermediário e avançado. Para projetos da disciplina, os itens básicos já fazem muita diferença e costumam ter

custo baixo de implementação, como permitir remapear controles e garantir que legendas tenham bom tamanho e contraste.

Um princípio importante em jogos é separar **o desafio pretendido** das **barreiras acidentais**. Um jogo de reflexo pode legitimamente exigir rapidez; mas exigir que o jogador perceba um perigo apenas pela cor, ou leia uma legenda minúscula, não faz parte do desafio. É uma barreira que pode ser removida sem tirar a graça do jogo.

10. Usabilidade, acessibilidade e a fronteira com UX

Usabilidade e acessibilidade são condições necessárias para uma boa experiência, mas não suficientes. Ambas fazem parte de um conceito mais amplo: a **experiência do usuário (UX)**.

Como os conceitos se relacionam

- **Acessibilidade** pergunta se *todas as pessoas* do público conseguem usar o sistema, incluindo pessoas com deficiência.
- **Usabilidade** pergunta se as pessoas conseguem usar o sistema *com eficácia, eficiência e satisfação* para atingir seus objetivos.
- **Experiência do usuário** abrange tudo isso e mais: as emoções, as expectativas, o valor percebido e o contexto, antes, durante e depois do uso.

As duas primeiras se sobrepõem bastante. Um site que não funciona com teclado tem um problema de acessibilidade e, para quem depende do teclado, também de usabilidade. Muitas boas práticas, como rótulos claros, bom contraste e mensagens de erro úteis, melhoram as duas ao mesmo tempo.

O que UX acrescenta

Um sistema pode ser acessível e usável e, mesmo assim, oferecer uma experiência ruim. Imagine um app de banco perfeitamente funcional que envia notificações de promoção várias vezes ao dia, ou um jogo fácil de jogar, mas sem nada que motive a voltar. A experiência também começa antes do uso (as expectativas criadas pela propaganda, as avaliações na loja) e continua depois (o suporte, o cancelamento, a lembrança que fica).

Esses aspectos serão aprofundados mais adiante na disciplina. Por ora, o importante é entender que usabilidade e acessibilidade são a base: sem elas, nenhuma outra qualidade da experiência consegue se sustentar.

Síntese, glossário e questões para reflexão

Síntese

- Para quem usa, a interface é o sistema. IHC estuda como projetar, avaliar e construir sistemas que as pessoas realmente conseguem usar.
- As pessoas agem com base em modelos mentais. A imagem do sistema é o único canal para comunicar um modelo coerente.
- Signifiers, mapeamento e feedback determinam se a interface comunica o que pode ser feito e o que aconteceu.
- Os golfos de execução e de avaliação ajudam a localizar onde a interação falha.
- Usabilidade depende de usuários, objetivos e contexto específicos, e se mede por eficácia, eficiência e satisfação.
- Avalia-se usabilidade por inspeção (especialistas) e por testes com usuários, abordagens que se complementam.
- Acessibilidade garante que pessoas com deficiência possam usar o sistema, beneficia muito mais gente e é obrigação legal no Brasil.
- As WCAG organizam a acessibilidade em quatro princípios: perceptível, operável, compreensível e robusto.
- Usabilidade e acessibilidade são a base da experiência do usuário, que será aprofundada mais adiante.

Glossário

Termo	Significado
Affordance	Relação entre objeto e pessoa que define quais ações são possíveis
ARIA	Conjunto de atributos que complementa o HTML com informações para tecnologias assistivas
Avaliação heurística	Método de inspeção em que especialistas verificam a interface com base em princípios gerais
Eficácia	Capacidade de concluir a tarefa com precisão
Eficiência	Recursos gastos (tempo, esforço) para concluir a tarefa

Termo	Significado
eMAG	Modelo de Acessibilidade em Governo Eletrônico
Feedback	Resposta do sistema que informa o resultado de uma ação
Golfo de avaliação	Dificuldade de perceber e interpretar o que o sistema fez
Golfo de execução	Dificuldade de descobrir como fazer o que se quer no sistema
Leitor de tela	Software que transforma o conteúdo da tela em fala ou braille
Mapeamento	Relação entre os controles e seus efeitos
Modelo mental	Representação interna que a pessoa constrói sobre como o sistema funciona
Pensar em voz alta	Protocolo em que o participante de um teste verbaliza o que pensa enquanto usa o sistema
Signifier	Sinal perceptível que comunica onde e como agir
SUS	System Usability Scale: questionário de 10 itens que mede a satisfação percebida
Tecnologia assistiva	Recurso que amplia a capacidade de pessoas com deficiência, como leitores de tela e acionadores
WCAG	Web Content Accessibility Guidelines, diretrizes de acessibilidade do W3C

Questões para reflexão

1. Pense em um sistema que você usa com frequência. Em que ponto o seu modelo mental dele é diferente do funcionamento real?
2. Escolha um app que você usa todo dia e identifique um signifier claro e um ausente ou ambíguo.

3. Lembre-se de uma mensagem de erro que te deixou sem saber o que fazer. Em qual golfo estava o problema? Como você a reescreveria?
4. Um sistema pode ter boa usabilidade para um público e má para outro? Dê um exemplo usando a definição da ISO 9241-11.
5. Navegue por um site que você já desenvolveu (ou que usa com frequência) usando apenas o teclado. O que funcionou e o que não funcionou?
6. Em um jogo que você conhece, quais desafios são parte da proposta e quais são barreiras que poderiam ser removidas?

Referências

- BARBOSA, Simone D. J.; SILVA, Bruno S. *Interação Humano-Computador*. Rio de Janeiro: Elsevier, 2010.
- BRASIL. Lei nº 13.146, de 6 de julho de 2015. Institui a Lei Brasileira de Inclusão da Pessoa com Deficiência (Estatuto da Pessoa com Deficiência).
- BRASIL. *eMAG: Modelo de Acessibilidade em Governo Eletrônico*. Governo Federal.
- GAME ACCESSIBILITY GUIDELINES. Disponível em: gameaccessibilityguidelines.com.
- IBGE. *PNAD Contínua: pessoas com deficiência, 2022*. Rio de Janeiro: IBGE, 2023.
- ISO 9241-11:2018. *Ergonomics of human-system interaction: Part 11: Usability: definitions and concepts*.
- MICROSOFT. *Inclusive Design Toolkit*.
- NIELSEN, Jakob. *Usability Engineering*. San Francisco: Morgan Kaufmann, 1993.
- NORMAN, Donald A. *O design do dia a dia*. Rio de Janeiro: Rocco.
- W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*. 2023.